ВЫЧИСЛИТЕЛЬНЫЕ МЕТОДЫ · ЛЕКЦИЯ 3

Основные численные методы

Ошибки, корни, линейные системы, интерполяция, дифференцирование и интегрирование

Лю Шисян

Кафедра теплофизики (Э6)
МГТУ им. Н.Э. Баумана
Москва / 2026

Место этой лекции в курсе



Лекция 3 — Основные численные методы

Численные методы позволяют получать приближённые решения математических задач с помощью конечной последовательности вычислений.

Задачи занятия:

- оценивать абсолютную и относительную погрешности;
- находить корни нелинейных уравнений;
- решать системы линейных алгебраических уравнений;
- выполнять интерполяцию, численное дифференцирование и интегрирование.

План

1. Численные вычисления и погрешности
2. Корни нелинейных уравнений
3. Системы линейных уравнений
4. Интерполяция и аппроксимация данных
5. Численное дифференцирование
6. Численное интегрирование

1

Численные вычисления и погрешности

Аналитическое и численное решение

Аналитическое решение

Решение можно представить в явном виде с помощью формулы.
Например:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

- результат получается напрямую;
- удобно исследовать влияние параметров;
- можно использовать для проверки численного решения.

Численное решение

Для многих задач удобной аналитической формулы не существует.
Например:

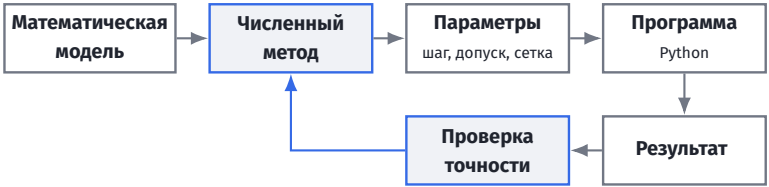
$$\cos x - x = 0.$$

Решение строится последовательностью вычислений:

$$x_0 \to x_1 \to x_2 \to \cdots \to \tilde{x}.$$

Необходимо выбрать метод, критерий остановки и требуемую точность.

Этапы численного расчёта



Проверка точности

Уменьшаем шаг или допуск и сравниваем результаты повторных расчётов.

Абсолютная и относительная погрешность

Точное значение $x = \sqrt{2}$; приближение $\tilde{x} = 1.4142$.

Абсолютная погрешность

$$\Delta x = |x - \tilde{x}| \approx 1.36 \cdot 10^{-5}.$$

Имеет те же единицы измерения, что и x.

Относительная погрешность

$$\delta x = \frac{|x - \tilde{x}|}{|x|} \approx 9.59 \cdot 10^{-6}.$$

Здесь это примерно $0.000959\,\%$.

```python
import numpy as np

exact_root = np.sqrt(2.0)
approx_root = 1.4142
absolute_error = abs(approx_root - exact_root)
relative_error = absolute_error / abs(exact_root)
```

При $x = 0$ относительная погрешность не определена: используем абсолютную.

Источники погрешности

Модель и данные

Исходные данные могут быть неточными, а математическая модель — упрощать реальную систему.

реальность $\rightarrow$ модель

Аппроксимация

Точная математическая операция заменяется приближённой.
Например:

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

Округление

Компьютер хранит числа с конечной точностью.
Поэтому арифметические операции также выполняются приближённо.

Ограниченная точность компьютера

Десятичная дробь хранится приближённо

```
value = 0.1 + 0.2
print(value)
print(value == 0.3)
```

Вывод

```
0.30000000000000004
False
```

Задаём допустимое отклонение

```
np.isclose(value, 0.3,
           rtol=0, atol=1e-15)
# True

np.finfo(float).eps
# 2.220446049250313e-16
```

`atol` — абсолютный допуск. Его выбирают по масштабу и точности задачи.

Для `float64` доступно примерно 15–16 значащих десятичных цифр.

Аппроксимация и выбор шага h

Численная производная

Заменяем производную конечной разностью:

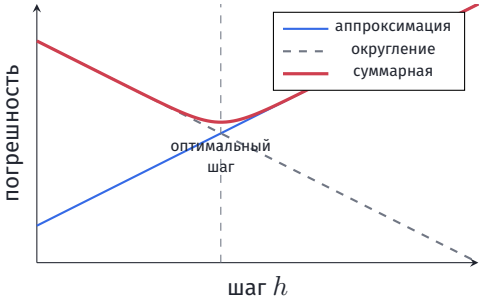
$$f'(x) \approx \frac{f(x+h) - f(x)}{h}.$$

Уменьшение h снижает ошибку аппроксимации.

Округление при малом шаге

При очень малом h вычитаются близкие числа: возрастает влияние округления.

Справа показана схема, а не результат расчёта.



2

Корни нелинейных уравнений

Задача поиска корня

Найти x^*, для которого $f(x^*) = 0$.

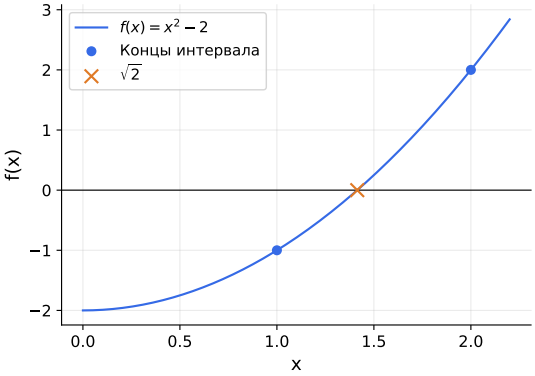
$$f(x) = x^2 - 2$$

На интервале $[1, 2]$:

$$f(1) = -1, \qquad f(2) = 2.$$

Проверочное значение

$$x^* = \sqrt{2} \approx 1.41421356.$$



Метод бисекции

Пусть функция непрерывна на отрезке $[a, b]$ и значения на концах имеют разные знаки:

$$f(a)\, f(b) < 0.$$

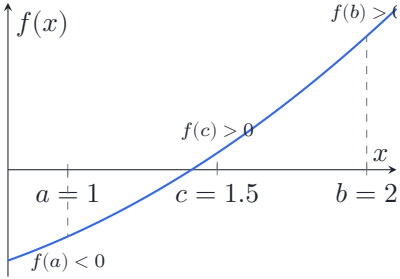
Тогда на интервале $[a, b]$ существует хотя бы один корень.

На каждом шаге берём середину интервала:

$$c = \frac{a + b}{2}.$$

Далее вычисляем $f(c)$ и оставляем ту половину, в которой сохраняется смена знака:

$$[a, b] \to [a, c] \quad \text{или} \quad [c, b].$$



Остановка

Для середины оставшегося интервала:

$$|c - x^*| \le \frac{b - a}{2} \le \varepsilon.$$

Метод бисекции на Python

```python
def bisection(f, a, b, tol=1e-8):
    if not (a < b and tol > 0):
        raise ValueError("Require a < b and tol > 0")
    if f(a) == 0: return a
    if f(b) == 0: return b
    if f(a) * f(b) > 0:
        raise ValueError("No sign change on [a, b]")
    while (b - a) / 2 > tol:
        c = (a + b) / 2
        if f(a) * f(c) <= 0:
            b = c
        else:
            a = c
    return (a + b) / 2
```

Интервал $[1, 2]$

```python
f = lambda x: x**2 - 2
root_bisect = bisection(
    f, 1.0, 2.0)
```

Результат

$x \approx 1.4142135605$

$|f(x)| \approx 5.24 \cdot 10^{-9}$

Допуск `tol` ограничивает половину длины интервала.

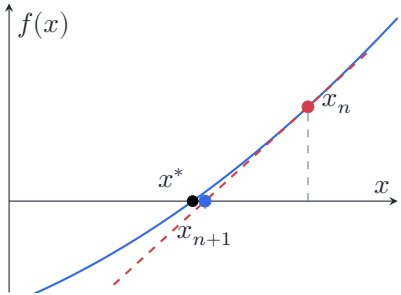
Метод Ньютона: идея

Строим **касательную** к графику функции в текущей точке x_n.
Точка пересечения касательной с осью x даёт новое приближение x_{n+1}.

На рисунке показан шаг из $x_n = 1.8$.

Формула метода Ньютона:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



Метод Ньютона на Python

```python
def newton(f, df, x0, tol=1e-8, max_iter=100):
    x = x0
    for _ in range(max_iter):
        if f(x) == 0: return x
        if df(x) == 0:
            raise ValueError("Zero derivative")
        x_new = x - f(x) / df(x)
        if abs(x_new - x) < tol:
            return x_new
        x = x_new
    raise RuntimeError("No convergence")
```

Начальная точка $x_0 = 1.5$

```python
f = lambda x: x**2 - 2
df = lambda x: 2*x
root_newton = newton(
    f, df, 1.5)
```

Результат

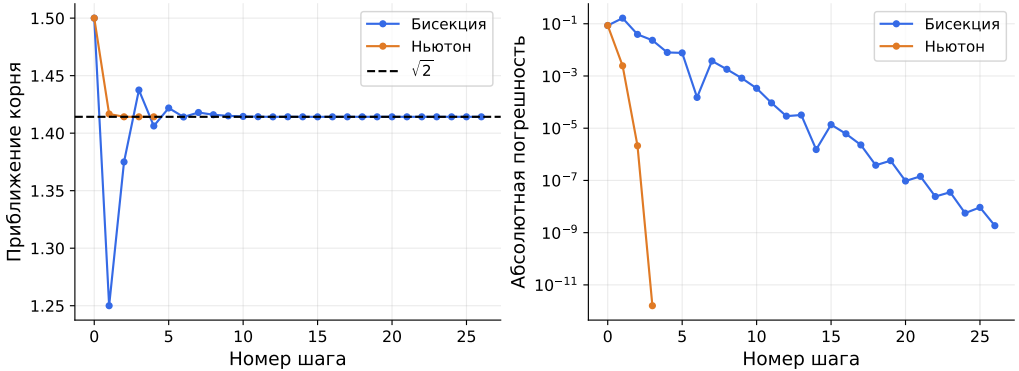
$x \approx 1.4142135624$

$|f(x)| \approx 4.44 \cdot 10^{-16}$

После остановки по шагу проверяем также невязку $f(x)$.

Как сходятся бисекция и метод Ньютона

$f(x) = x^2 - 2$: бисекция на $[1, 2]$, Ньютон из $x_0 = 1.5$; `tol=1e-8`.



Бисекция сужает интервал вдвое. Ньютон быстро сходится вблизи простого корня.

Готовые методы: scipy.optimize

```python
from scipy.optimize import root_scalar

result_bisect = root_scalar(
    f, bracket=[1, 2], method="bisect",
    xtol=1e-8)

result_newton = root_scalar(
    f, fprime=df, x0=1.5,
    method="newton", xtol=1e-8)

print(result_newton.root)
print(result_newton.converged)
```

Вывод

```
1.4142135623730951
True
```

Бисекция: непрерывность и смена знака на концах.

Ньютон: нужна производная; результат зависит от x_0.

`converged` сообщает об остановке метода; невязку проверяем отдельно.

3

Системы линейных уравнений

Система линейных уравнений

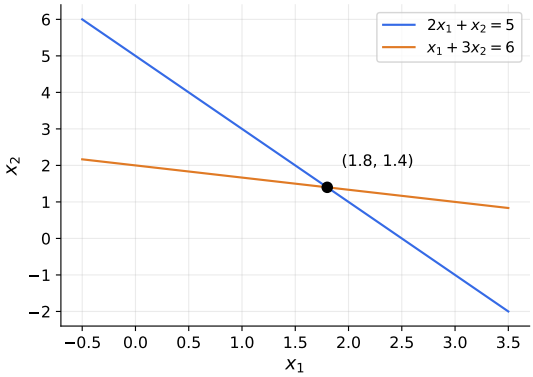
$$\begin{cases} 2x_1 + x_2 = 5, \\ x_1 + 3x_2 = 6. \end{cases}$$

Матричная запись: $A\mathbf{x} = \mathbf{b}$.

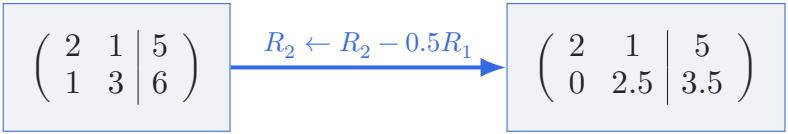
$$A = \begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 5 \\ 6 \end{pmatrix}.$$

Точка пересечения

$x_1 = 1.8, \quad x_2 = 1.4.$



Метод Гаусса: исключаем неизвестную




```
A = np.array([[2., 1.], [1., 3.]])
b = np.array([5., 6.])
augmented = np.column_stack((A, b)).copy()
factor = augmented[1, 0] / augmented[0, 0]
augmented[1] -= factor * augmented[0]
```

Обратный ход

$$x_2 = \frac{3.5}{2.5} = 1.4$$

$$x_1 = \frac{5 - x_2}{2} = 1.8$$

В этом примере перестановка строк не нужна. В общем алгоритме выбирают ведущий элемент.

Решение системы в NumPy

```
A = np.array([[2., 1.],
              [1., 3.]])
b = np.array([5., 6.])

linear_solution = np.linalg.solve(A, b)
linear_residual = A @ linear_solution - b

print(linear_solution)
print(linear_residual)
```

Решение

```
[1.8 1.4]
```

Невязка $Ax - b$

```
[0. 0.]
```

`solve` решает квадратную невырожденную систему.

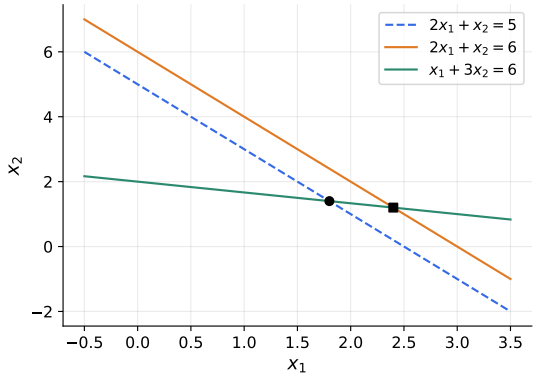
Явно вычислять обратную матрицу для решения системы не требуется.

Изменение правой части

В первом уравнении заменим 5 на 6.

```
b_changed = np.array([6.,
    6.])
x_changed = np.linalg.solve(
    A, b_changed)
print(x_changed)
# [2.4 1.2]
```

Первая прямая сдвигается параллельно себе. Точка пересечения меняется.



Малая невязка и чувствительность решения

```
A_sensitive = np.array([
    [1., 1.], [1., 1. + 1e-6]])
b_sensitive = A_sensitive @ np.array([1., 1.])

x_before = np.linalg.solve(
    A_sensitive, b_sensitive)
x_after = np.linalg.solve(
    A_sensitive, b_sensitive + [0., 1e-6])

print(np.linalg.cond(A_sensitive))
# about 4e6
```

До возмущения

$\mathbf{x} \approx (1, 1)$

После возмущения

$\mathbf{x} \approx (0, 2)$

Изменение правой части на 10^{-6} заметно меняет решение.

Большое число обусловленности означает чувствительность к данным. Малая невязка сама по себе не гарантирует точности решения.

4

Интерполяция и аппроксимация данных

Интерполяция и подбор параметров

Интерполяция

Строим функцию, проходящую через заданные точки.

- линейная: отрезки между соседними узлами;
- кубический сплайн: гладкие кусочно-кубические участки.

Подбор параметров

Выбираем параметры модели по всем данным.

- например, прямая $y = ax + b$;
- совпадение со всеми точками не требуется.

Один набор данных для сравнения

```
x = np.array([0., 1., 2., 3., 4.])
y = np.array([0.2, 1.1, 1.8, 3.2, 3.9])
x_new = np.linspace(0, 4, 200)
```

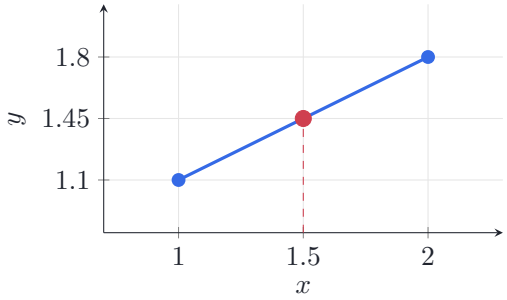
Линейная интерполяция

Между соседними узлами используем прямую.

$$y(x) = y_i + \frac{x - x_i}{x_{i+1} - x_i}(y_{i+1} - y_i).$$

```
y_lin = np.interp(x_new, x, y)
print(np.interp(1.5, x, y))
# 1.45
```

Здесь 1.5 — середина между 1 и 2, поэтому берём среднее 1.1 и 1.8.



Кубический сплайн

```python
from scipy.interpolate import CubicSpline

spline = CubicSpline(x, y)
y_spline = spline(x_new)

print(spline(x))
# [0.2 1.1 1.8 3.2 3.9]
```

Между узлами

На каждом интервале — кубический полином.

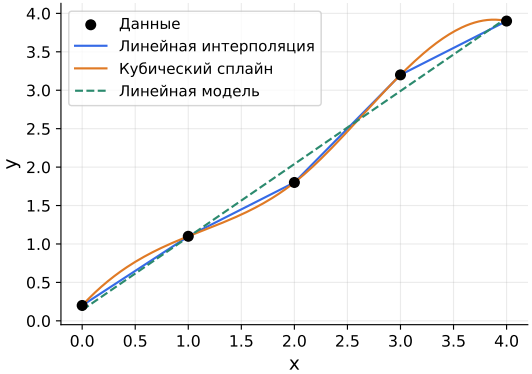
В узлах

Согласованы значения, первая и вторая производные.

`CubicSpline` по умолчанию использует краевые условия `not-a-knot`.

Гладкость кривой не означает, что она точнее описывает физический процесс.

Одни данные: три способа построить кривую



Линейная интерполяция и сплайн проходят через узлы. Прямая МНК описывает общую зависимость.

Метод наименьших квадратов

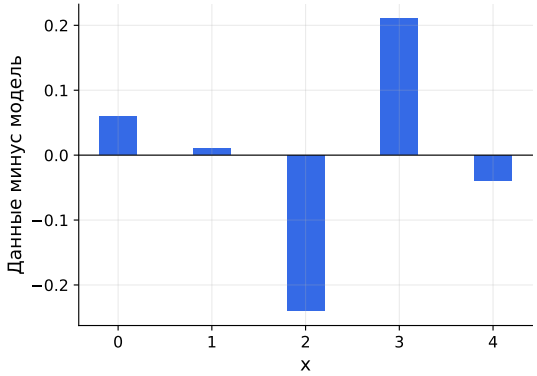
Подбираем прямую $y = ax + b$ с наименьшей суммой квадратов отклонений.

```
a, b = np.polyfit(x, y, 1)
y_fit = a * x_new + b
residuals = y - (a*x + b)
rmse = np.sqrt(
    np.mean(residuals**2))
```

Результат

$a = 0.95, \quad b = 0.14$

RMSE ≈ 0.1463



Температура между точками измерения

```
x_T = np.array([0., 1., 2., 3., 4.])

T = np.array([300., 315., 327.,
              346., 358.])

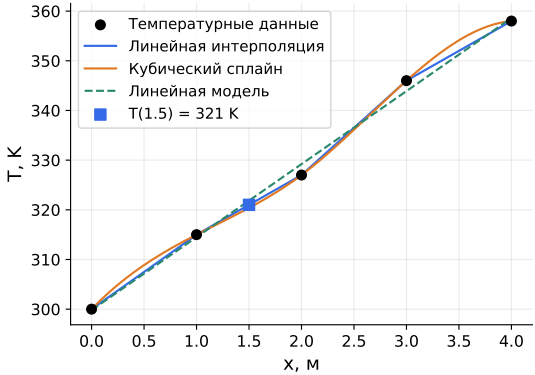
T_at_15 = np.interp(1.5, x_T, T)
T_spline = CubicSpline(x_T, T)
a_T, b_T = np.polyfit(x_T, T, 1)
```

x_T — м; T — К.

При $x = 1.5$ м:

линейная: 321 К; сплайн: 320.375 К.

Прямая МНК: $T \approx 14.7x + 299.8$.



5

Численное дифференцирование

Производная из дискретных данных

Определение производной

Производная определяется как

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

Но в численных расчётах h всегда конечно. Поэтому производную заменяют **конечной разностью**.

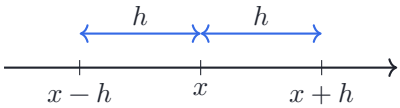
Разность вперёд

$$\boxed{f'(x) \approx \frac{f(x+h) - f(x)}{h}}$$

Центральная разность

Используем значения с двух сторон от точки:

$$\boxed{f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}}$$



Порядок точности разностных формул

Из разложения Тейлора получаем:

Разность вперёд

$$\frac{f(x+h)-f(x)}{h} = f'(x) + \frac{h}{2}f''(x) + O(h^2).$$

Ведущая ошибка пропорциональна h.

$$h \,/\, 10 \implies E \,/\, 10$$

Центральная разность

$$\frac{f(x+h)-f(x-h)}{2h} = f'(x) + \frac{h^2}{6}f^{(3)}(x) + O(h^4).$$

Ведущая ошибка пропорциональна h^2.

$$h \,/\, 10 \implies E \,/\, 100$$

Эти оценки работают для достаточно гладкой функции, пока преобладает ошибка аппроксимации.

Численная производная на Python

$f(x) = \sin x, \quad x = 1, \quad h = 10^{-3}; \qquad f'(1) = \cos 1 \approx 0.5403023059.$

```
f = np.sin
x = 1.0
h = 1e-3

df_forward = (f(x+h) - f(x)) / h
df_central = (f(x+h) - f(x-h)) / (2*h)
df_exact = np.cos(x)

err_forward = abs(df_forward - df_exact)
err_central = abs(df_central - df_exact)
```

Разность вперёд

0.5398814804

Погрешность $4.21 \cdot 10^{-4}$.

Центральная разность

0.5403022158

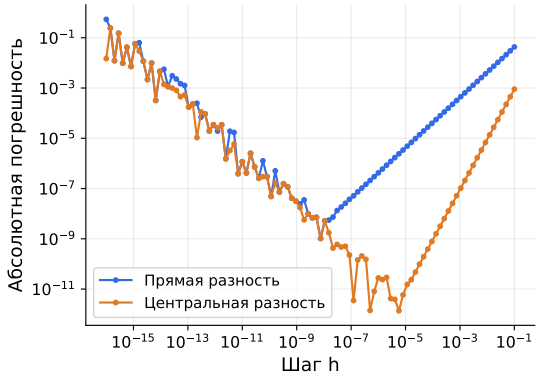
Погрешность $9.01 \cdot 10^{-8}$.

Как шаг влияет на погрешность производной

$f(x) = \sin x, x = 1.$

Меняем h от 10^{-16} до 10^{-1} и сравниваем с $\cos 1$.

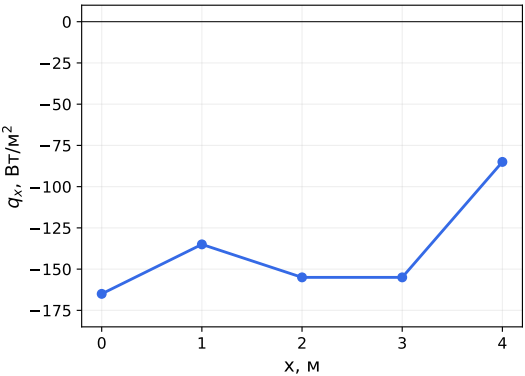
При малом h вычитаются близкие числа, и округление начинает преобладать.



Из температурного профиля — к тепловому потоку

$$q_x = -\lambda \frac{dT}{dx}, \quad \lambda = 10 \text{ Вт/(м К)}.$$

```
conductivity = 10.0
temperature_gradient = np.gradient(
    T, x_T, edge_order=2)
heat_flux = (
    -conductivity *
     temperature_gradient)
```



`gradient`: центральные разности внутри сетки; односторонние второго порядка на краях.

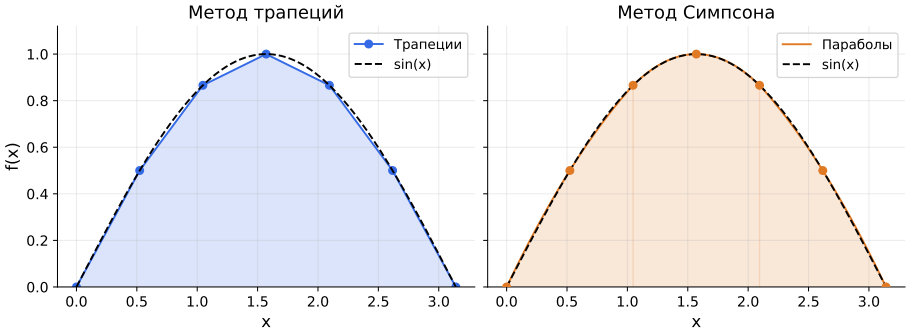
Температура растёт с x: $q_x < 0$, тепло переносится в сторону уменьшения x.

6

Численное интегрирование

Интегрирование: прямые или параболы

$$I = \int_0^{\pi} \sin x \, dx = 2.$$



6 интервалов, 7 узлов. Трапеция использует 2 соседних узла; парабола — 3.

Формулы трапеций и Симпсона

Равномерная сетка: $h = (b-a)/N$, $f_i = f(x_i)$.

Трапеция: один интервал

$$\int_{x_i}^{x_{i+1}} f(x)\,dx \approx \frac{h}{2}(f_i + f_{i+1}).$$

Складываем площади всех трапеций.

Для достаточно гладкой функции общая погрешность $O(h^2)$.

Симпсон: два интервала

$$\int_{x_0}^{x_2} f(x)\,dx \approx \frac{h}{3}(f_0 + 4f_1 + f_2).$$

Складываем вклады пар интервалов; N должно быть чётным.

Для достаточно гладкой функции общая погрешность $O(h^4)$.

При уменьшении h вдвое ожидаем снижение ошибки примерно в 4 и 16 раз, пока не мешает округление.

Численное интегрирование в Python

```
from scipy.integrate import simpson

N = 100
x_int = np.linspace(0, np.pi, N + 1)
y_int = np.sin(x_int)

I_trap = np.trapezoid(y_int, x_int)
I_simp = simpson(y_int, x=x_int)
I_exact = 2.0

print(I_trap)
print(I_simp)
```

Трапеции

$I \approx 1.9998355039$

$|I - 2| \approx 1.64 \cdot 10^{-4}.$

Симпсон

$I \approx 2.0000000108$

$|I - 2| \approx 1.08 \cdot 10^{-8}.$

100 интервалов — 101 узел.

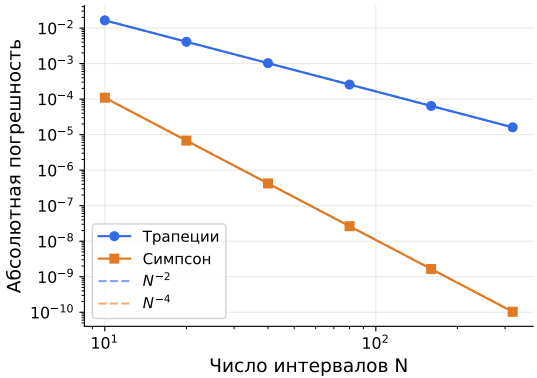
Проверяем точность сгущением сетки

Для $\sin x$ на $[0, \pi]$ точный интеграл равен 2.

Число интервалов:

$10, 20, 40, 80, 160, 320.$

На каждой сетке вычисляем $|I_N - 2|$.



Средняя температура по длине образца

Используем те же измерения $T(x)$ на отрезке $0 \le x \le 4$ м.

$$\overline{T} = \frac{1}{L}\int_0^L T(x)\,dx, \qquad L = 4 \text{ м}.$$

```
length = x_T[-1] - x_T[0]
mean_trap = np.trapezoid(T, x_T) / length
mean_simpson = simpson(T, x=x_T) / length

print(mean_trap)
print(mean_simpson)
```

Трапеции

$\overline{T} = 329.25$ К.

Симпсон

$\overline{T} \approx 329.67$ К.

Разница около 0.42 К показывает зависимость от метода. Точный профиль между измерениями неизвестен, поэтому это не точная оценка ошибки.

Одна функция: корень, производная и интеграл

$$g(x) = x^3 - x - 1$$

```
g = lambda x: x**3 - x - 1
root = root_scalar(
    g, bracket=[1, 2], method="brentq").root

h = 1e-4
derivative = (g(root+h) - g(root-h)) / (2*h)

x = np.linspace(1, 2, 101)
I_trap = np.trapezoid(g(x), x)
I_simp = simpson(g(x), x=x)
```

Корень и производная

$x^* \approx 1.3247179572$

$g'(x^*) \approx 4.264633.$

Интеграл на $[1, 2]$

Трапеции: $1.250075.$

Симпсон: $1.25.$

Точное значение: $1.25.$

Итоги лекции

- **Корни и линейные системы:** находим решение, проверяем невязку и чувствительность к данным.
- **Данные:** различаем интерполяцию через узлы и подбор параметров модели.
- **Производная и интеграл:** выбираем шаг и проверяем, как меняется результат на более мелкой сетке.

Код занятия

github.com/lyushisyan/computational-methods-course